

2026-09-22 - Security Lab Log

今日の目的

CSIRT / Incident Responseを学習するための仮想環境を構築し、Kali LinuxからUbuntu Serverへの通信を実際に観測する。

今日は特に、

- 仮想ネットワークの仕組みを理解する
- NATとHost-onlyの違いを確認する
- Nmapでポートを調査する
- tcpdumpでNmapの通信を見る
- SYN / SYN-ACK / RSTを理解する

ことを目標にした。

1. セキュリティラボの構築

VMware Workstation上に、

- Kali Linux
- Ubuntu Server
- Windows 11

を用意した。

Ubuntu Serverは現在、

```
CPU      : 2 vCPU
RAM      : 6 GB
Storage  : 40 GB
```

で構成している。

UbuntuをLinuxサーバー、Kaliを攻撃・診断側、Windows 11をクライアントとして使用する。

2. ネットワークについて調査

最初はVMからインターネットへ接続できない問題を調査した。

`ip route`を確認すると、Host-only環境では外部へ出るための default routeが存在していないことが分かった。

NATへ変更すると、

```
default via 192.168.160.2 dev ens33
```

が追加された。

Ubuntuには、

```
192.168.160.131
```

が割り当てられ、

```
ping 8.8.8.8
```

も成功した。

ここで **192.168.160.2** は別のPCではなく、VMwareが用意しているNAT Gatewayだと理解した。

3. Host-only Network

セキュリティ検証時はHost-onlyへ戻した。

現在の検証ネットワークは、

```
192.168.159.0/24
```

Ubuntu:

```
192.168.159.129  
ens33
```

Kali:

```
192.168.159.131  
eth0
```

となっている。

Host-onlyは「VM同士も通信できない」という意味ではなく、

```
Kali <----> Ubuntu
```

は通信可能で、

```
Lab ----X----> Internet
```

のように外部ネットワークから分離できることを理解した。

4. 調査ツールの導入

Ubuntu Serverに調査用ツールを導入した。

主なものは、

```
nmap
tcpdump
tshark
curl
wget
git
net-tools
traceroute
dnsutils
lsof
htop
jq
rsyslog
auditd
```

など。

今後、ネットワーク調査だけでなく、Linuxログやプロセスの調査にも使用する予定。

5. Nmapによるポートスキャン

KaliからUbuntuへSYN Scanを実施した。

```
sudo nmap -sS 192.168.159.129
```

結果、

PORT	STATE	SERVICE
22/tcp	open	ssh

となった。

代表的な1000 TCPポートのうち、999ポートがclosedで22/tcpだけがopenだった。

UbuntuではSSH Serverが動作しているため、22番ポートが応答している。

6. tcpdumpでNmapを観測

次に、

「Nmapを実行すると、対象サーバーからは何が見えるのか？」

を確認した。

Ubuntu側で、

```
sudo tcpdump -i ens33 -nn \  
'host 192.168.159.131 and tcp'
```

を実行。

その状態でKaliから、

```
sudo nmap -sS -p 22 192.168.159.129
```

を実行した。

7. 実際に見えたパケット

tcpdumpでは、

```
Kali -> Ubuntu:22      Flags [S]  
Ubuntu:22 -> Kali      Flags [S.]  
Kali -> Ubuntu:22      Flags [R]
```

という通信を確認できた。

流れにすると、

Kali	Ubuntu
SYN	
----->	
SYN-ACK	
<-----	
RST	
----->	

SYN

Flags [S]

「TCP接続を開始したい」という要求。

SYN-ACK

Flags [S.]

SYNを受け取った側からの応答。

今回NmapはこのSYN-ACKを受け取ったことで、

22/tcp open

と判断している。

RST

Flags [R]

接続をリセットするためのTCPフラグ。

通常のTCP接続では、

SYN -> SYN-ACK -> ACK

で接続を成立させる。

しかしNmapの `-sS` はSYN Scanなので、

SYN -> SYN-ACK -> RST

となった。

SYN-ACKを受け取った時点でOPENだと分かるため、完全なTCP接続を成立させず終了している。

今日分かったこと

今日一番大きかったのは、

「Nmapで **22/tcp open** と表示される裏側で何が起きているのか」

を実際のパケットで確認できたこと。

今まではNmapの結果だけを見ていたが、tcpdumpで対象側から見ることで、

```
Nmap
  ↓
TCP SYN
  ↓
Server
  ↓
SYN-ACK / RST
  ↓
NmapのOPEN/CLOSED判定
```

という関係が見えるようになった。

また、

- IPアドレス
- サブネット
- デフォルトゲートウェイ
- NAT
- Host-only
- TCPポート
- SYN
- SYN-ACK
- RST

が、それぞれ独立した知識ではなく、実際の通信としてつながっていることも確認できた。

次の疑問

tcpdumpを見ればNmapのスキャンを確認できる。

では、

「人間がtcpdumpを見ていなくても、サーバー側でポートスキャンを自動検知できるのか？」

という疑問が出てきた。

次は、

```
Kali
|
| 大量のSYN
v
```

```
Ubuntu
|
v
Detection
```

という流れを作りたい。

同一IPから短時間に多数のポートへSYNが送られた場合に、それを異常として検知できるか検証する。

Next

次回は、

- OPEN / CLOSEDポートのパケット比較
- tcpdumpフィルタの理解
- SSH通信の観測
- Linuxログの確認
- ポートスキャン検知

へ進む予定。